

SDEV 2351

DAY 1: NESTED ROUTES, ROUTE PARAMETERS, AND ROUTE CONSTRAINTS

MODULE 1 — ROUTING & STATE

Session Objectives

1. Define nested route hierarchies in a Blazor application
2. Use route parameter syntax with type-specific route constraints
3. Design multi-level navigation patterns (GitHub/Jira-style)

Today's Schedule

PHASE	TIME	WHAT'S HAPPENING
Connecting	10 min	Routing bridge from SDEV 2301
Exploring	30 min	Nested routes, parameters, constraints + Admin demonstration
Practicing	50 min	Two hands-on practices (guided + independent)
Assessing	10 min	Knowledge check + exit ticket

PHASE 1 · 10 MINUTES

CONNECTING

From SDEV 2301 basics to enterprise routing

Why Nested Routes?

Real-world URLs encode **resource hierarchies**:

```
github.com/{org}/{repo}/tree/{branch}/{path}
```

- Each segment identifies a different resource
- Flat routing breaks down at scale -- 50 pages with no structure
- Nested routes organize pages into logical sections

You already know `@page`, `NavLink`, and simple `{id:int}` from SDEV 2301. Today we go deeper.

PHASE 2 · 30 MINUTES

EXPLORING

Nested hierarchies, parameters, constraints — then a live demonstration

Nested Route Hierarchies

Layout chains solve section-level navigation:

- `MainLayout` wraps `AdminLayout` wraps page component
- `@layout` directive assigns a layout to a page
- `@inherits LayoutComponentBase` + `@Body` renders child content
- `_Imports.razor` in a folder applies `@layout` to all pages in that folder

Key idea: Layouts nest like folders — each level adds its own chrome (sidebar, header) around the content below it.

Route Parameters and Constraints

Expand from basic `{id:int}` to the full constraint system:

CONSTRAINT	EXAMPLE	MATCHES
<code>int</code>	<code>{id:int}</code>	<code>42</code>
<code>bool</code>	<code>{active:bool}</code>	<code>true</code> , <code>false</code>
<code>guid</code>	<code>{key:guid}</code>	Valid GUID strings
<code>minlength</code>	<code>{name:minlength(3)}</code>	3+ characters
<code>regex</code>	<code>{code:regex(^[A-Z]{2}\$)}</code>	Two uppercase letters

Chain constraints with colons: `{id:int:min(1)}`

Constraint Failure Behavior

- Constraint failure = **no route match** (404)
- It is **not** an exception -- no error page, no crash
- The framework simply does not match the route

```
@page "/products/{id:int:min(1)}"

// /products/42    → match
// /products/abc   → 404 (not an int)
// /products/0     → 404 (fails min(1))
```

[Full constraint list: see reference-sheet.md](#)

Demonstration: Admin Section

Watch the nested layout chain built live in `Day01Demo` — **7 directives**, no files created:

- `AdminLayout.razor` — `@inherits LayoutComponentBase`, `@layout MainLayout`, `@Body`
- `Admin/_Imports.razor` — `@layout AdminLayout` scopes the layout to the folder
- Three admin pages — one `@page` directive each

The render chain is `MainLayout → AdminLayout → page`. You build the same pattern yourself in Practice 1.

PHASE 3 · 50 MINUTES

PRACTICING

Two practices — guided + independent

Practice 1: Reports Section (worked launch, 20 min)

The same pattern the demonstration just showed — green theme

- `ReportsLayout.razor` — the two opening directives, plus the content placeholder
- `Reports/_Imports.razor` — the folder-level layout directive
- Three report pages — the directive that gives each one a route
- Verify the layout chain renders both sidebars

Same 7 directives as the Admin demonstration — recall them from what you just watched. Stuck? The Practice Walkthrough names which directive each file needs.

Practice 2: Product Catalog with Dual Constraints (Independent, 25 min)

Independent practice — the day's second pattern

- Add a route: `{category:alpha}/{id:int:min(1)}`
- Add two `[Parameter]` properties for each segment
- Test valid/invalid URLs:
 - `/products/electronics/42` -- matches
 - `/products/home-garden/42` -- fails `:alpha` (hyphens rejected)
 - `/products/electronics/0` -- fails `:min(1)`

Practice 3: Project Management Section

Take-home extension — for additional practice after class

- Combine both patterns: nested layouts + parameterized routes
- Build a project management section with its own layout
- Routes include project ID and section parameters

Days 2–3 build on Practices 1 and 2 only — this extension is optional. A code reveal page is published on the day's Brightspace page.

Common Troubleshooting

ISSUE	CAUSE	FIX
Layout not rendering	Missing <code>@inherits LayoutComponentBase</code>	Add the directive to your layout
Pages ignore layout	No <code>_Imports.razor</code> in folder	Add <code>@layout <SectionLayout></code> to the folder's <code>_Imports.razor</code>
Parameter always null	Missing <code>[Parameter]</code> attribute	Decorate the property
<code>:alpha</code> rejects hyphens	<code>:alpha</code> matches <code>[A-Za-z]</code> only	Use <code>:regex</code> for hyphenated strings

PHASE 4 · 10 MINUTES

ASSESSING

Summary

Today you learned to:

1. **Define nested route hierarchies** using layout chains, `@layout`, and `_Imports.razor`
2. **Use route parameters with constraints** like `int`, `bool`, `guid`, `minlength`, and `regex`
3. **Design multi-level navigation** patterns inspired by GitHub and Jira URL structures

Exit Ticket

Sketch a 3-level route hierarchy for an HR app (e.g., `/hr/employees/{id}`).
Name the layout at each level and one route constraint you'd apply.

What's Next

Day 2: Multiple route parameters, `NavigationManager`, and programmatic navigation

- Combine several parameters in one route template
- Use `NavigateTo()` to navigate in code
- Build wizard-style multi-step flows

No pre-class reading for Day 2. Day 2 builds directly on today's routes and constraints. The first required reading lands at the end of Day 2 (~56 min, due Day 3) -- bridges your SDEV 2301 events and delegates knowledge to the Fluxor pattern we start on Day 3.